

Python Data Structure

Formative Activity

Update your e-portfolio with your thoughts, outcomes from the seminar and your team discussions

This reflection considers how factors programmers may need to consider to justify their choice of data structures. To examine this point, the following scenario will be explored.

Scenario

Smart MAT is a small independent company, working with schools in the UK to provide data driven solutions to school improvement. Schools often find it challenging to consider the progress that pupils have made in a variety of subjects which can, in turn, be a barrier to evaluating the quality of education across a range of departments. Smart MAT has developed a data driven model to consider evaluating progress for pupils, given their starting points and the qualification they have studied in a secondary school setting.

The data needed to complete this model is clear:

1. What defined the pupils starting point?
2. How are pupils expected to progress in different qualifications?

When considering any evaluation of evidence, it is important to consider any established industry standards. There is a recognised starting point for pupils in a secondary setting based on their Key Stage 2 SAT (national exams taken at age 11). The Department for Education (DfE) publish the Transition Matrices (TM) which show how many pupils secure different grades based on their starting points. An example for GCSE Maths is shown below:

Figure 1: Transition Matix - Maths

KS2 Attainment	1	2	3	4	5	6	7	8	9	u	x	Covid impacted	All Grades
Total	32721	48797	74479	114744	121059	75349	59307	44696	25119	10498	5069	0	611838
Less than 80	1505	839	482	272	159	53	37	22	6	1172	172	0	4719
80 - 89.5	7643	4874	2341	946	288	50	22	3	3	3340	629	0	20139
90 - 95.5	8686	11321	9843	5635	1924	354	143	43	3	1886	827	0	40665
96 - 99.5	5288	10710	15309	14623	7035	1424	545	149	23	965	772	0	56843
100 - 102	2644	6795	13220	18362	12248	3235	1211	312	41	525	582	0	59175
102.5 - 104.5	1890	5301	12934	24033	21715	7937	3398	1048	195	448	598	0	79497
105 - 107	1119	3106	9066	22134	27137	13996	7427	3067	636	248	487	0	88423
107.5 - 109.5	455	1351	4321	13617	22783	17377	12176	6613	1727	134	279	0	80833
110 - 113	209	560	1901	7558	16976	18588	18163	13838	5716	74	237	0	83820
113.5 - 116.5	30	63	302	1362	4287	7226	10477	12422	8812	12	71	0	45064
117 - 120	1	4	22	93	399	923	1988	3831	5471	0	10	0	12742
No scaled score	3251	3873	4738	6109	6108	4186	3720	3348	2486	1694	405	0	39918

Since September 2024, the DfE allow programmers access to data via Python using PANDAS Data Frames to convert raw csv files into a more usable data format. This contains additional columns such as the percentage of pupils achieving a grade.

Complications With Data Formats

The is imported as a data frame using the PANDAS module and then converted to a Python dictionary. This helps to illustrate the issue with data formats, as all values are now stored as strings.

Pupils starting points are a decimal value form 0 to 120. However the transition matrices group the starting points into bands, represented by a text string. Helpfully, in nearly all of the bands the information is in a common format: *lower – upper*. However there is an exception with the lowest band, ‘Less than 80’.

All GCSEs follow a common grading system of an integer grade form 1 to 9 or a grade U, apart from GCSE combined science. GCSE combined science covers the same grade range, but it is possible to achieve additional grades of 2-1, 3-2, 4-3,...9-8 which correspond to points values 1.5, 2.5, 3.5...8.5. This is because, as the name suggests the GCSE combines the equivalent of 2 GCSEs in science. This means the Python code has to deal with an additional data structure.

Python Code

The Python Code in Appendix 1, takes an integer starting point (ks2) and a subject as a string and applied the following algorithm:

1. Match the subject with the TM dictionary.
2. Match the KS2 Band by considering the limits specified in the band string and the ks2 decimal value.
3. Calculate the expected value given the subject and band
4. Calculate the probability of achieving a threshold grade
5. Return the results as a tuple

The results are returned as a tuple so that this code can be imported as a module to any wider application that requires this information.

Summary

This example show how data structures and formats are used to manipulate data is a real world scenario.

References

DfE (2024). *Explore education statistics: Data catalogue*. Available at: [Data catalogue - Explore education statistics - GOV.UK](#) [Accessed 20 July 2025]

OpenDSA (no date). *1.1. Data Structures and Algorithms*. OpenDSA. Available at: [1.1. Data Structures and Algorithms — OpenDSA Data Structures and Algorithms Modules Collection](#) [Accessed 20 July 2025]


```

#calculates probabilities as a decimal from the sting stored
in TM

chance9 = float(tm['perc_9'][i])/100
chance98 = chance9 + float(tm['perc_8'][i])/100
chance97 = chance98 + float(tm['perc_7'][i])/100
chance96 = chance97 + float(tm['perc_6'][i])/100
chance95 = chance96 + float(tm['perc_5'][i])/100
chance94 = chance95 + float(tm['perc_4'][i])/100
chance93 = chance94 + float(tm['perc_3'][i])/100
chance92 = chance93 + float(tm['perc_2'][i])/100
chance91 = chance92 + float(tm['perc_1'][i])/100
else:
    #caclulates the output for combined-science which has a
different data structure
    expected = round((float(tm['perc_11'][i])*1
        + float(tm['perc_21'][i])*1.5
        + float(tm['perc_22'][i])*2
        + float(tm['perc_32'][i])*2.5
        + float(tm['perc_33'][i])*3
        + float(tm['perc_43'][i])*3.5
        + float(tm['perc_44'][i])*4
        + float(tm['perc_54'][i])*4.5
        + float(tm['perc_55'][i])*5
        + float(tm['perc_65'][i])*5.5
        + float(tm['perc_66'][i])*6
        + float(tm['perc_76'][i])*6.5
        + float(tm['perc_77'][i])*7
        + float(tm['perc_87'][i])*7.5
        + float(tm['perc_88'][i])*8
        + float(tm['perc_98'][i])*8.5
        +
float(tm['perc_99'][i])*9
        )/100,2)

    #caluates cumulative probabilities of achieving a grade
    chance9 = round(float(tm['perc_99'][i])/100,3)
    chance98 = round((chance9 + (float(tm['perc_98'][i]) +
float(tm['perc_88'][i]) )/100),3)
    chance97 = round((chance98 + (float(tm['perc_87'][i]) +
float(tm['perc_77'][i]) )/100),3)
    chance96 = round((chance97 + (float(tm['perc_76'][i]) +
float(tm['perc_66'][i]) )/100),3)
    chance95 = round((chance96 + (float(tm['perc_65'][i]) +
float(tm['perc_55'][i]) )/100),3)
    chance94 = round((chance95 + (float(tm['perc_54'][i]) +
float(tm['perc_44'][i]) )/100),3)
    chance93 = round((chance94 + (float(tm['perc_43'][i]) +
float(tm['perc_33'][i]) )/100),3)

```

```

        chance92 = round((chance93 + (float(tm['perc_32'][i]) +
float(tm['perc_22'][i]) )/100),3)
        chance91 = round((chance92 + (float(tm['perc_21'][i]) +
float(tm['perc_11'][i]) )/100),3)

    #displays probabilities of achieving threshold grades
    print('Chance 7+ ' + str(chance97*100) + '%')
    print('Chance 5+ ' + str(chance95*100) + '%')
    print('Chance 4+ ' + str(chance94*100) + '%')
    #returns statistics as a tuple
    return [expected, round(chance91,3), round(chance92,3),
round(chance93,3), round(chance94,3),
            round(chance95,3), round(chance96,3), round(chance97,3),
round(chance98,3), round(chance9,3), band]

#Only executed if the script is run as a main programme
if __name__ == "__main__":
    #runs the code with test data
    subject='Art and Design'
    #subject = 'Combined Science'
    ks2=102
    get_expected(subject,ks2)

```